

Практическая Работа Тема 1 Архитектура ЭВМ. Гарвардская архитектура на примере МК AVR. Основы программирования на языке Assembler.

Микроконтроллер - это микропроцессорная система, выполненная на одном кристалле (в одной микросхеме, в одном корпусе) и содержащая процессор, память и разнообразные устройства ввода - вывода данных. Микроконтроллеры содержат все составные элементы вычислительной машины, поэтому их также называют «микро-ЭВМ». Микроконтроллеры в настоящее время являются основой для разработки и проектирования автоматизированных устройств.

Микроконтроллеры AVR выпускаются фирмой Microchip после покупки фирмы Atmel (Advanced Technology Memory and Logic). Быстродействующее RISC - ядро AVR разработали студенты университета из Норвегии Альф Боген и Вергард Волен. Имена разработчиков и архитектура ядра (Alf, Vergard, RISC) определили название микроконтроллеров - AVR. Архитектура оказалась достаточно удачной и определила дальнейшее развитие семейства микроконтроллеров.

Работу микропроцессорной системы позволяет пояснить обобщенная функциональная схема микро-ЭВМ или микроконтроллера AVR (рисунок 1.1).

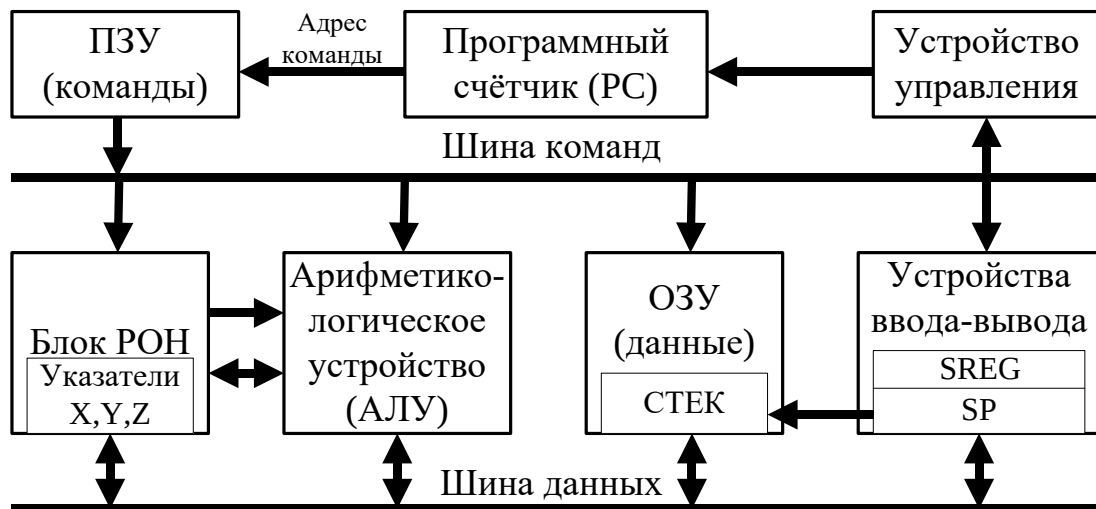


Рисунок 1.1 Обобщённая функциональная схема микроконтроллера

1 этап. Выдача адреса команды на память команд. Программа хранится в **памяти команд**. В микроконтроллерах обычно используют постоянное запоминающее устройство (ПЗУ), так как в процессе работы программа не изменяется. Адрес выполняемой команды хранится в **программном счетчике** (Program Counter - РС). На выходе памяти команд после подачи адреса появляется код выполняемой команды.

2 этап. Декодирование команды. Команды хранятся в памяти команд в виде двоичных кодов. Код команды содержит отдельные части (их называют полями). Это поля кода операции (что делать?) и адресной части

(где находятся операнды?). Для указания места расположения операндов (или аргументов) используют различные методы адресации. Укажем основные.

1) Регистровая адресация. В команде указаны один или два регистра, в которых находятся операнды. Это основной метод адресации, используемый при обработке данных, расположенных в рабочих регистрах. Он работает с регистрами r0 – r31 (Рисунок 1.1, группа регистров 1).

2) Непосредственная адресация. В команде содержатся данные. (Рисунок 1.1, группа регистров 2). Непосредственная адресация работает только с регистрами r16 – r31.

3) Косвенная адресация. В команде определен указатель адреса X, Y, Z (двухбайтовый регистр), содержащий адрес ячейки памяти, в которой находится операнд. (Рисунок 1.1, группа регистров 3).

4) Прямая адресация. Команда содержит адрес операнда.

При описании регистров общего назначения необходимо упомянуть регистры r0 и r1 (Рисунок 1.1, группа регистров 4) – в них записывается результат операций умножения, более подробно в теме 3.

3 этап. Выполнение команды. Производится в арифметико-логическом устройстве (АЛУ) совместно с рабочими общего назначения (РОН). АЛУ – набор операционных блоков. Регистры общего назначения (другое название - рабочие регистры) - устройства для хранения операндов и результатов выполняемых операций. Блок РОН выполнен как двухадресная (двухпортовая) память, он содержит 32 регистра разрядностью 8 бит (или 1 байт). Из блока РОН на входы АЛУ одновременно поступает содержимое двух регистров. Это операнды, участвующие в выполняемой операции. Результат записывается вместо первого операнда. Содержимое второго регистра сохраняется.

В АЛУ формируются признаки результата выполненной операции, которые записываются в регистр состояния процессора (SREG – Status Register). Все пересылки данных выполняются через рабочие регистры.

4 этап. Запись результата операции. Результат операции обязательно записывается в определенное запоминающее устройство. При регистровой адресации результат записывается в РОН, указанный в команде после мнемоники. При косвенной или прямой адресации результат может быть записан в ОЗУ или в регистры устройств ввода/вывода.

5 этап. Формирование адреса следующей команды выполняется устройством управления. Если программа не содержит переходов (или ветвлений), то код программного счетчика изменяется линейно. После выполнения команды он увеличивается на 2, так как команды в AVR – двухбайтовые (1 команда в памяти занимает 16 бит). Адреса команд в программном счетчике всегда четные (0, 2, 4...). Адресуется младший байт команды.

Логическая организация ОЗУ данных микроконтроллеров AVR отображает использование адресов общего пространства для обращения к

запоминающим устройствам различного назначения и физического исполнения.

Оперативная (временная, энергозависимая память) представлена модулем SRAM (Static Random Access Memory – статическая память с произвольным доступом). Каждая ячейка выполнена как триггер на транзисторах, что обеспечивает высокую скорость чтения/записи.

Разрядность ОЗУ данных микроконтроллера AVR Atmega328P составляет 1 байт. Емкость зависит от разрядности адреса N. Максимальную емкость, равную 2^N ячеек, называют общим адресным пространством.

Для общего адресного пространства ОЗУ данных (рисунок 1.1) предусмотрен адрес разрядностью 2 байта, что обеспечивает адресацию до 64 Кбайт ячеек памяти. Часть адресов общего пространства использована для адресации регистров. Такое соответствие характеризуют фразой: «регистры отображаются на память». Результатом такого отображения является возможность адресовать регистр как ячейку памяти.

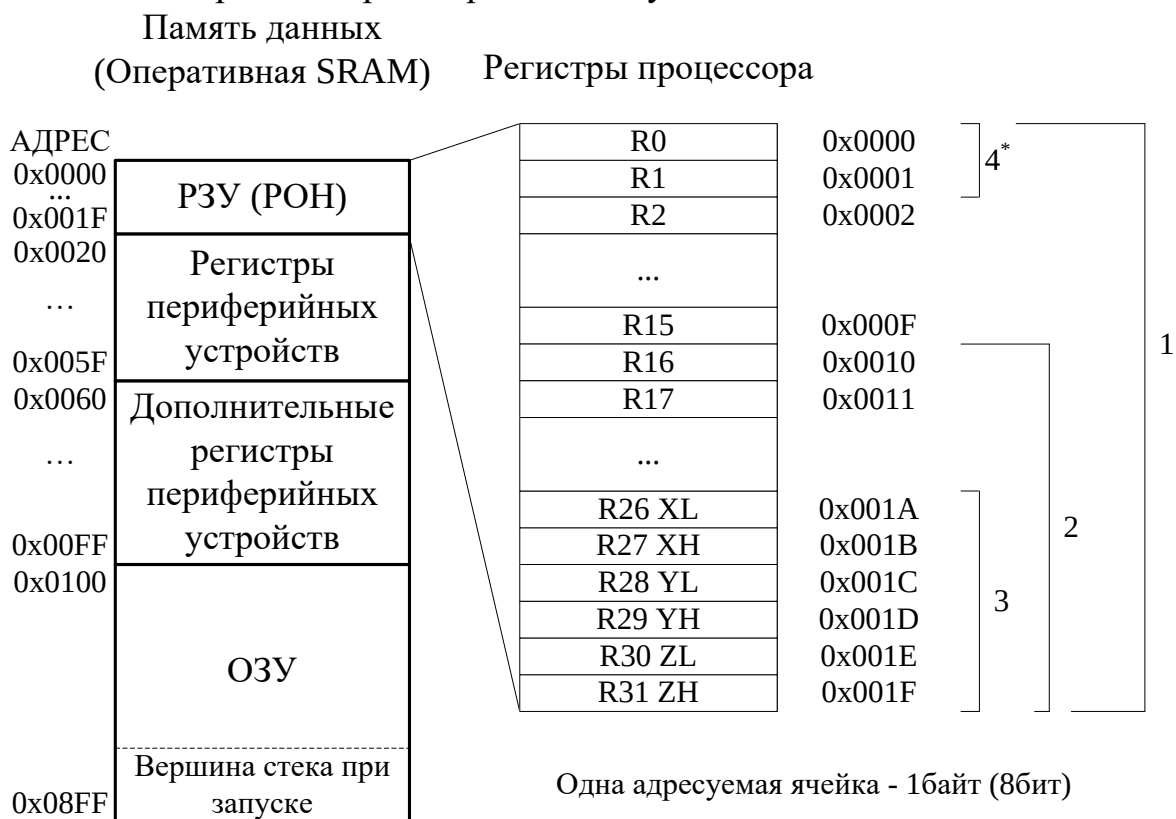


Рисунок 1.2 – организация оперативной памяти

1. Сегмент регистров общего назначения (РОН) или рабочих регистров содержит 32 регистра, каждый из которых непосредственно подключен к арифметико-логическому устройству (АЛУ) и способен выполнять функции аккумулятора. Все арифметические и логические операции, а также все пересылки между другими сегментами общего пространства, выполняются с использованием РОН.

2. Второй сегмент (64 адреса) отведен под регистры устройств ввода – вывода. Эти регистры предназначены для связи процессоров с интерфейсными устройствами, они содержат сигналы различных типов:

данные (Data), сигналы управления (Control), сигналы режима (Mode), сигналы состояния (Status). Все эти сигналы определенным образом размещены в регистрах устройств ввода/вывода. Они имеют индивидуальные уникальные имена (и адреса), приводятся в Datasheet (техническое описание). Операции со всеми регистрами сегмента можно производить по их именам с использованием команд in, out, sbi, cbi.

3. Третий сегмент (160 адресов) – дополнительные регистры периферийных устройств. Данный сегмент есть не у всех контроллеров семейства и введен для увеличения количества периферийных устройств. Обращение к регистрам возможно только через команды lds, sts.

4. Сегмент 4 содержит адреса ячеек памяти внутреннего (физически реализованного в кристалле) ОЗУ, которое имеет определенную емкость, зависящую от конкретной модели.

Задание 1.1. Исследуйте выполнение команд пересылки с различными методами адресации, приведенных в программе Example 1 (см. Таблицу 1.1).

Программа Example 1 написана на ассемблере. Ассемблер AVR подробно описан в Help и представлен в приложении, использует стандартные правила оформления программ. Его отличия от ассемблера процессоров Intel обусловлены спецификой аппаратных средств. Ассемблер AVR **не чувствителен к регистру**, на практике чаще используют нижний регистр. Важное значение в языках уровня Ассемблера имеет отступ от начала строки. Метки принято ставить в начале строки, команды с небольшим отступом. Однако в Microchip (Atmel) studio данное условие имеет только рекомендательный характер.

Таблица 1.1 – Описание программы Example 1.

; Example 1	mov r1, r16 ;14	push r1 ;28
.include "m328pdef.inc";1	mov r2, r17 ;15	push r2 ;29
ldi r16, 0x08 ;2	mov r3, r18 ;16	push r3 ;30
out sph, r16 ;3	mov r4, r18 ;17	push r4 ;31
ldi r16, 0xff ;4	in r4, portd ;18	pop r14 ;32
out spl, r16 ;5	ldi xh, 0x01 ;19	pop r13 ;33
m: call pp1 ;6	ldi xl, 0 ;20	pop r12 ;34
; call pp2 ;7	st x+, r1 ;21	pop r11 ;35
; call pp3 ;8	st x+, r2 ;22	ret ;36
jmp m ;9	st x+, r3 ;23	pp2: ;37
pp1: ldi r16, 10 ;10	st x+, r4 ;24	ret ;38
ldi r17, 0x10 ;11	ld r10, -x ;25	pp3: ;39
ldi r18, 0x20 ;12	sts 0x0110, r16;26	ret ;40
ldi r19, 0x30 ;13	lds r5, 0x0100 ;27	

Признак начала комментария - точка с запятой (;). В комментариях допускается русский шрифт. Символ начала комментария позволяет при отладке временно исключить из компиляции строку программы. В программах номера строк записаны как комментарии (для ссылок).

Директивы ассемблера начинаются с точки, определяют условия компиляции. В строке 1 записана директива `.include "m328pdef.inc"`, которая подключает к программе файл конфигурации, содержащий имена внутренних регистров и соответствующие им адреса для конкретной модели ATmega328p, что позволяет использовать в программе символические имена регистров, указателей и портов. Содержимое файла можно посмотреть в любом текстовом редакторе. В Microchip Studio Version 7.0.2542 данный файл расположен в каталоге: `C:\Program Files (x86)\Atmel\Studio\7.0\Packs\atmel\ATmega_DFP\1.6.364\avrasm\inc`.

Инициализация указателя стека (строк 2 - 5) необходима для работы со стеком. В указатель стека, который содержит два регистра – `sph` и `spl` (Stack Pointer high, Stack Pointer low) записываются два байта, определяющие максимальный адрес ОЗУ (`RamEnd`). Стек располагается в конце оперативной памяти. Первая ячейка стека после инициализации – последняя ячейка ОЗУ, а каждое новое значение записывается в ячейку с меньшим адресом.

В строках 2-5 используются функций ассемблера `«low»` и `«high»`, результаты которых – константы, равные соответственно младшему или старшему байту константы `ramend`. Максимальный адрес физической памяти `«Ramend»`, указан в файле конфигурации, для модели ATmega328P он равен `0x08ff`.

В программе инициализация приведена для примера, среда Microchip выполняет эту процедуру автоматически, т.е. эти строки можно опустить.

Использование подпрограмм. Сложные программы собирают из подпрограмм, каждая из которых имеет в начале метку, являющуюся именем подпрограммы (например, `pp1:`, `pp2:`, `pp3:`), а в конце – команду возврата из подпрограммы `«ret»`. В основной программе должен быть инициализирован стек, который используется процессором для хранения адреса возврата из подпрограммы.

Обращение к подпрограммам: `call <метка>`

Начало подпрограммы: `<метка>:`

Завершение подпрограммы: `ret`

Подпрограммы можно отключать в процессе отладки, записывая в начале строки вызова подпрограммы символ комментария `(;)`.

При вызове подпрограмм адрес команды, расположенной за командой `«call»` (его называют адресом возврата), сохраняется в стеке, а значение указателя стека уменьшается на 2. Заметим, для хранения адреса и каждой команды требуется 2 байта. (При записи данных в стек с использованием команд `push` и `pop` адрес будет меняться на 1.) При возврате из подпрограммы этот адрес извлекается из стека и загружается в счетчик команд. Значение указателя стека соответственно увеличивается на 2.

В рамках данных работ подпрограммы удобно использовать для реализации отдельных заданий, чтобы не создавать новый проект для каждого задания или примера.

В данной программе в строках 6-8 записаны команды вызова подпрограмм. При запуске программы будет выполняться только вызов первой подпрограммы. Остальные временно закомментированы, они имеют в начале строки точку с запятой. После исследования первой подпрограммы необходимо запретить ее вызов и разрешить работу другой подпрограммы. Строки 1 - 9 программы Example 1 будут использоваться впоследствии как начало других программ.

Программы микроконтроллеров часто содержат **бесконечный цикл**, который многократно повторяется. Команда `jmp` (строка 9) выполняет безусловный переход на начало пользовательской программы, отмеченное меткой `m`.

Исследуемая программа начинается с метки `pr1`. Программа содержит примеры команд пересылки с различными методами адресации. Методы адресации определяют, каким образом в адресной части команды указано расположение данных.

Команды пересылки данных используются для инициализации вычислительных процессов и для определенного размещения исходных данных в регистрах микроконтроллера, в ОЗУ данных, а также в регистрах периферийных устройств. На примерах команд пересылки рассмотрим методы адресации микроконтроллера AVR.

Мнемоника команд с **непосредственной адресацией** содержат букву «i» от «Immediate» - непосредственно (строки 10 - 13). В строке 10 записана команда пересылки с непосредственной адресацией. Первый операнд – регистр получатель, второй – константа, мнемоника. Систему счисления указывает префикс: `0x` (или `$`) - 16-ричная; `0b` - двоичная; префикса нет - десятичная.

! Во всех командах на первом после мнемоники месте указан получатель результата.

Команда пересылки с регистровой адресацией: `mov r1, r16`.

Первый регистр - получатель, второй регистр – источник (строки 14 - 17). Операнды могут быть размещены в любом РОН - `r0` – `r31`.

Команда записи в регистр периферийных устройств из регистра с прямой адресацией порта: **`out sph, r16` мнемоника `out`**. На первом месте указан адрес порта (или соответствующее имя), куда выводятся данные, а на втором – регистр источник, откуда берутся данные (строки 3 и 5). **Вывод в порт возможен только из регистра.**

Для пересылки константы в порт эту константу вначале записывают в регистр командой с непосредственной адресацией для временного хранения. Обычно это регистр `r16` (строки 2 и 4).

Команда загрузки в РОН из регистра ПУ:

`in r4, porta` (строка 18). Во всех командах на первом месте указан получатель, а на втором – источник данных.

Команды обращения к памяти

Обращение к памяти - это загрузка регистра из памяти (мнемоника `ld` от слова `Load` - загрузить), или запись в память из регистра (`st` – `Store`,

сохранить) – выполняют команды с косвенной адресацией различных модификаций. При **косвенной адресации** в команде указывается адрес любой ячейкой адресного пространства, к которой происходит обращение. В системе команд предусмотрено три указателя адреса операнда - X, Y, или Z. Каждый указатель содержит два РОН (X состоит из XL(R26) и XH(R27), Y состоит из YL(R28) и YH(R29), Z состоит из ZL(R30) и ZH(R31)) и позволяет адресовать до 64 Кбайт памяти. Это адресное пространство ОЗУ данных, содержащее адреса ячеек памяти и всех регистров. В командах обращения к памяти после мнемоники на первом месте указывают получатель данных, а на втором – источник. Это регистр (r0 – r31) и ячейка памяти, для которой записывают имя указателя, содержащего адрес.

При использовании команд с косвенной адресацией начале выполняется инициализация указателя (строки 19, 20), а затем – пересылка. В командах с косвенной автоинкрементной адресацией (строки 21 - 24) вначале выполняется обращение к ячейке памяти по адресу указателя, а затем – инкремент (+1) этого адреса. В командах с косвенной автодекрементной адресацией (строка 2) вначале выполняется декремент (-1), а затем обращение к ячейке по адресу указателя.

Команды обращения к памяти с прямой (абсолютной) адресацией (строки 26, 27) содержат 2-байтовый адрес операнда (0x0000 – 0xffff в 16-ричной системе счисления). Адреса 0x0000 – 0x001f позволяют обращаться к РОН; адреса 0x0020 – 0x005f соответствуют регистрам внешних устройств; адреса 0x0060 – 0x00ff соответствуют дополнительным регистрам внешних устройств; с адреса 0x0100 начинается ОЗУ данных.

Команды обращения к стеку - push (загрузка содержимого регистра в стек) и pop (извлечение в регистр из стека) (строки 28 - 35). Стек реализован как участок системной памяти, адресуемый указателем SP, работающий как буфер типа LIFO. Стек используется для сохранения и последующего восстановления содержимого регистров при выполнении подпрограмм и операций обработки данных.

При записи данных в стек вначале выполняется декремент (-1) содержимого SP, а затем запись содержимого регистров в стек. При чтении действия выполняются в противоположном порядке: вначале - чтение, а затем инкремент SP.

Отладка программы в системе Microchip (Atmel) Studio

Создание и настройка проекта

Запустите среду Microchip (Atmel) Studio (начальное окно представлено на рисунке 1.3). В главном меню программы выберите File/New/Project (или сочетание клавиш Ctrl+Shift+N). В появившемся окне (см. рисунок 1.4) выберите Assembler, AVR Assembler Project. Укажите имя проекта и рабочую директорию. В аудиториях кафедры путь должен начинаться следующим образом D:/Users/№группы. В пути должны присутствовать только английские символы. Затем нажмите ОК. В появившемся окне (см. рисунок 1.5) выберите контроллер Atmega328P. Нажмите ОК.

В результате будет создан проект и открыто окно рабочего файла для ввода текста программы на ассемблере (с расширением .asm).

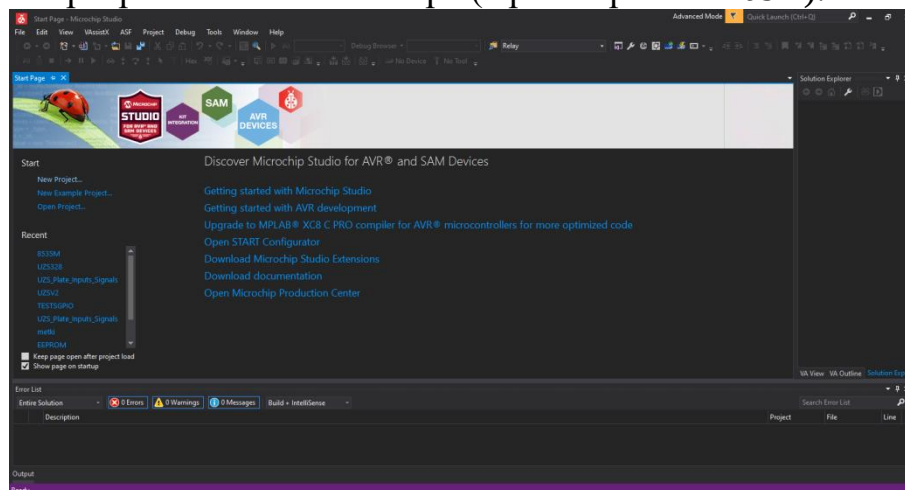


Рисунок 1.3 – Начальное окно программы

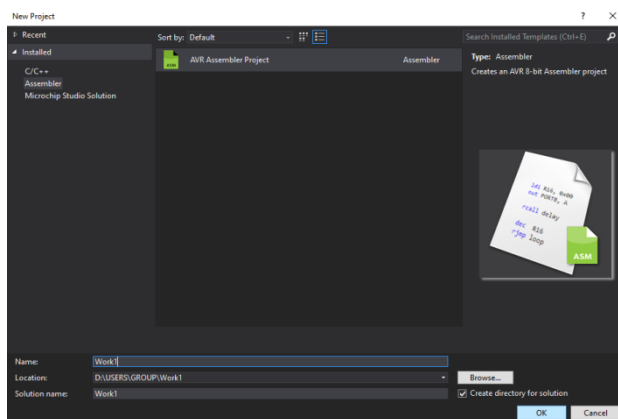


Рисунок 1.4 – Выбор типа проекта

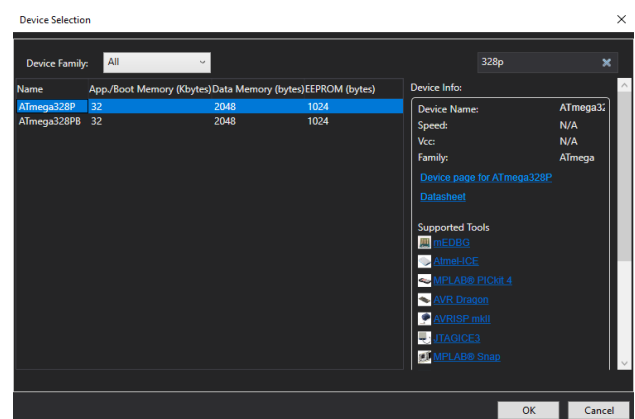


Рисунок 1.5 – Выбор целевого контроллера

После ввода программы необходимо выбрать в главном меню Project/Properties/ (сочетание клавиш Alt+F7). В появившемся окне (рисунок 1.6) выбрать вкладку Tool. В выпадающем списке Select debugger/Programmer выбрать Simulator.

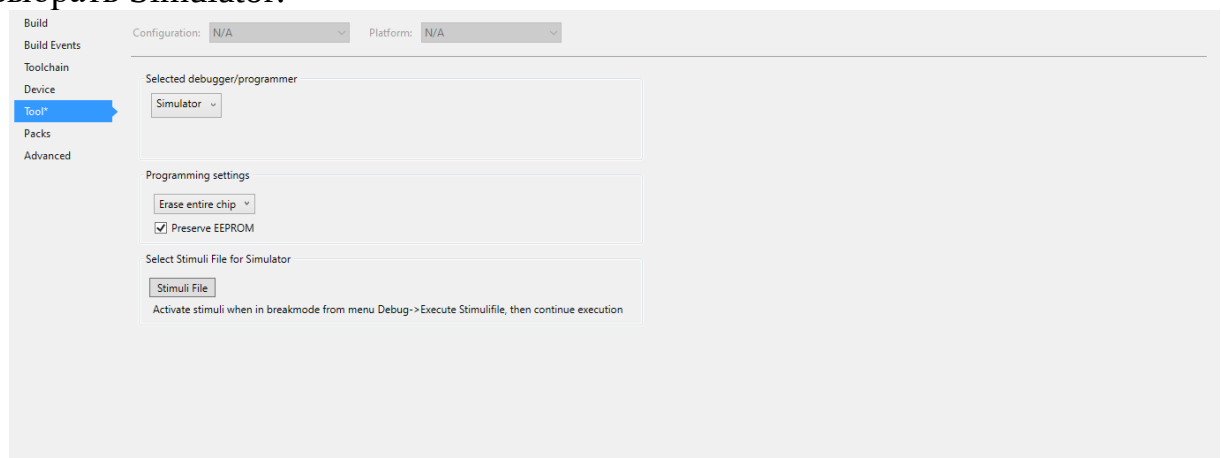


Рисунок 1.6 – Выбор отладчика

Для сборки проекта необходимо в главном меню выбрать Build/Build Solution или нажав клавишу F7. При отсутствии ошибок (предупреждения допускаются) можно запустить отладку программы из меню Debug/Start Debugging and Break, или кнопкой «Play/Pause». Для пошаговой отладки необходимо нажать F11. Окно отладчика представлено на рисунке 1.7.

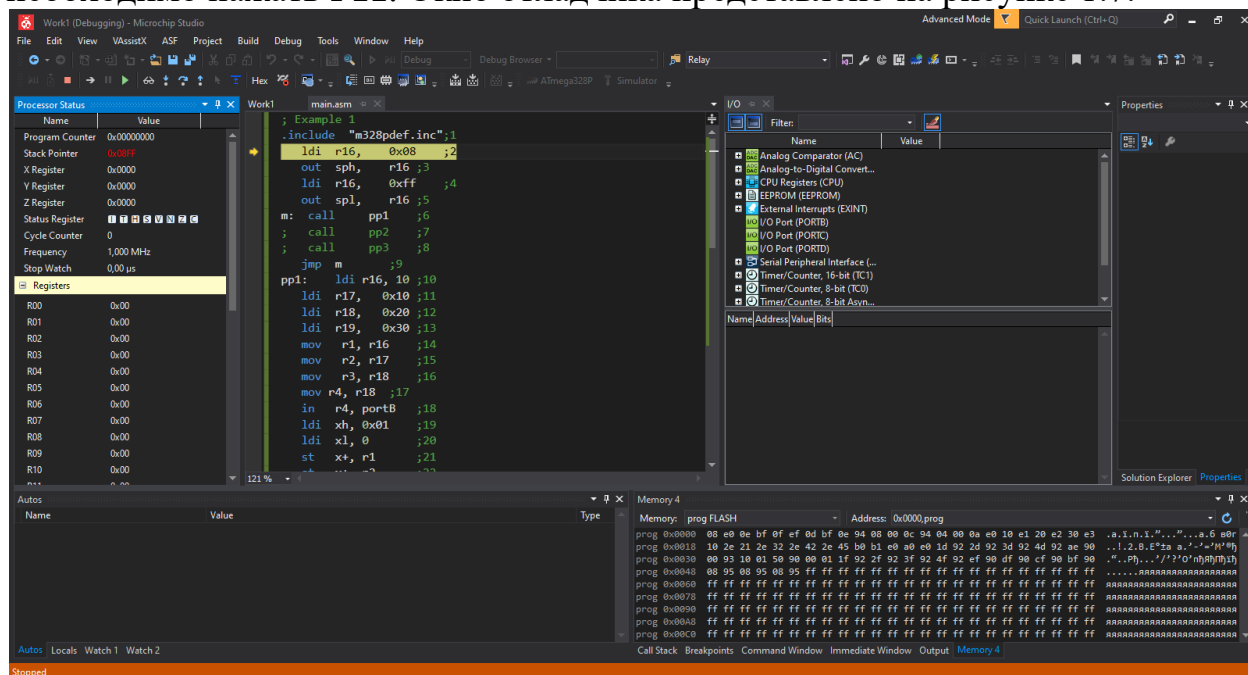


Рисунок 1.7 – Режим отладки

В режиме отладки, выбирая из меню Debug/Windows, можно открыть несколько панелей инструментов и окон, например, Processor, I/O View, Memory.

В окне Processor (левая панель на рисунке 1.7) отображается содержимое системных регистров – программного счетчика, указателя стека, указателей для косвенной адресации (X,Y,Z), временные параметры выполнения программы, регистр признаков результата «SREG», указатели стека, косвенных адресов, программный счетчик.

Вкладка «Memory» – позволяет выполнять отладку работы с памятью. Для работы с окном «Memory» необходимо указать тип отображаемой памяти, установить количество колонок таблицы. Вид всех окон настраивается через контекстное меню, открываемое правой кнопкой мыши. Можно выбрать размер шрифта, систему счисления для отображаемых данных. Для анализа выполнения логических операций, выполняемых поразрядно, удобна 16-ричная система, а для арифметических операций – десятичная.

Содержание отчета

Выполняйте программу Example 1 по шагам, нажимая клавишу F11. Описание функциональное назначение окон, использованных при работе с программой и типа информации, отображаемой в этих окнах. Опишите функции директивы (строка 1), инициализацию стека (строки 2-5) и вызовы подпрограмм (строки 6-8), назначение бесконечного цикла. Результаты

выполнения подпрограмм представьте в виде таблиц, содержащих номер строки программы, команду, описание выполняемого действия и результат – содержимое модифицируемых регистров и ячеек памяти.

В таблице можно использовать различные обозначения и сокращения, например символы «:=» обозначают присваивание; запись $M(0100)$ - содержимое ячейки памяти с адресом 0x0100, $M(SP)$ - содержимое памяти по адресам указателя стека SP.

В последующих работах, в отчётах для команд сдвига и арифметических операций будет необходимо выполнить проверку результатов с использованием калькулятора, и привести значения операндов и результата в десятичной и в двоичной системах счисления.

Пример заполнения трассы выполнения программы представлен в таблице 1.2.

Таблица 1.2 – Пример заполнения трассы в отчёте

№	Команда	Действие	Результат
10	ldi r16, 10	$r16:=10$ в r16 загрузить 10	$r16=0x0a$
11	ldi r17, 0x10	$r17:=0x10$ в r16 загрузить 10_{16}	$r17=0x10$
12	ldi r18, 0x20	$r18:=0x20$ в r16 загрузить 20_{16}	$r18=0x20$
13	ldi r19, 0x30	$r19:=0x30$ в r16 загрузить 30_{16}	$r19=0x30$
14	mov r1, r16	$r1:=(r16)$ в r1 переслать содержимое r16.	$r1=0x0a$
15	mov r2, r17	$R2:=(r17)$ в r2 переслать содержимое r17.	$r2=0x10$

Задание 1.2. Составьте подпрограмму pr2, для которой заданы числа A,B,C,D (варианты заданий см. таблицу 1.3):

Таблица 1.3 – Исходные данные по вариантам для задания 1.2

Бригада	1	2	3	4	5	6	7	8	9	10	11	12
A	160	212	30	45	570	65	85	99	110	130	125	725
B	125	30	210	780	100	300	65	301	500	360	225	400
C	360	270	221	45	300	200	400	80	180	80	450	55
D	500	770	450	21	450	100	200	220	720	20	80	35

Запишите заданные числа в ячейки памяти с адресами 0x0100, 0x0101, 0x0102, 0x0103. Для чисел, заданных в десятичной системе счисления, приведите 16-ричные значения. Опишите особенности выполнения операций с предложенными исходными данными в отчёте.

Задание 1.3. Составьте подпрограмму pr3, которая сохраняет заданные числа в стеке, извлекает числа из стека в заданные регистры (варианты заданий см. таблицу 1.4).

Таблица 1.4 – Исходные данные по вариантам для задания 1.3

Бригада	1	2	3	4	5	6	7	8	9	10	11	12
A	r0	r1	r5	r9	r5	r3	r5	r4	r6	r9	r9	r9
B	r1	r2	r3	r9	r6	r4	r3	r5	r7	r6	r7	r8
C	r2	r3	r2	r8	r7	r5	r2	r3	r8	r8	r8	r7
D	r3	r4	r7	r7	r8	r6	r1	r2	r9	r7	r6	r6

Контрольные вопросы

1. Опишите функциональный состав микроконтроллеров AVR.
2. Опишите формат строки ассемблерной программы.
3. Какие функции выполняют регистры общего назначения?
4. Поясните функциональное назначение команд пересылки.
5. Какие команды обеспечивают использование подпрограмм?
6. Как осуществляется инициализация стека? Как работает стек?
7. Опишите команды с непосредственной адресацией, назначение, примеры записи.
8. Опишите команды с регистровой адресацией, назначение, примеры записи.
9. Опишите команды с косвенной регистровой адресацией, назначение, примеры записи.
10. Что такое «адресное пространство», как определяется его размер?